

Refactoring: Separación de Instrumentos del Sistema y Personalizados

Fecha: 2026-05-24

Rama: main

Commits relevantes: [e240a5e](#) (checkpoint merge Pili), [a0dc603](#) (refactoring)

Contexto y motivación

Antes de este cambio, la tabla `instrumento` mezclaba dos tipos de instrumentos en una sola tabla, distinguiéndolos por una columna `usuario_id`:

- `usuario_id = NULL` → instrumento del sistema (cargado con fixtures, disponible para todos)
- `usuario_id != NULL` → instrumento creado por un usuario concreto

El tutor indicó que esta estructura era poco limpia ("chapuza") y recomendó separar ambos tipos en tablas distintas para que la responsabilidad de cada tabla fuera clara.

Estructura anterior

```
instrumento
└── id
    nombre
    usuario_id (NULL = sistema, NOT NULL = personalizado)

instrumento_musico
└── id
    musico_id (FK → musico)
    instrumento_id (FK → instrumento)
```

`instrumento_musico` era una entidad PHP manual con su propio repositorio. Las relaciones se gestionaban a mano en el controlador (persist/remove por cada fila).

Estructura nueva

<pre>instrumento_sistema └── id nombre</pre>	<pre>instrumento_personalizado └── id nombre usuario_id (FK → usuario)</pre>
--	--

musico_instrumento_sistemamusico_id (FK → musico)
instrumento_sistema_idmusico_instrumento_personalizadomusico_id (FK → musico)
instrumento_personalizado_id

Las dos tablas de relación (`musico_instrumento_sistema` y `musico_instrumento_personalizado`) las gestiona Doctrine automáticamente mediante `ManyToMany`. Ya no existe ninguna entidad PHP para la tabla intermedia.

Ficheros creados

Fichero	Descripción
<code>src/Entity/InstrumentoSistema.php</code>	Entidad para instrumentos del sistema (id, nombre)
<code>src/Entity/InstrumentoPersonalizado.php</code>	Entidad para instrumentos de usuario (id, nombre, usuario)
<code>src/Repository/InstrumentoSistemaRepository.php</code>	Repositorio de InstrumentoSistema
<code>src/Repository/InstrumentoPersonalizadoRepository.php</code>	Repositorio de InstrumentoPersonalizado
<code>migrations/Version20260524120000.php</code>	Migración que crea las nuevas tablas y migra los datos existentes
<code>docs/refactoring-instrumentos.md</code>	Este documento

Ficheros eliminados

Fichero	Motivo
<code>src/Entity/Instrumento.php</code>	Sustituido por InstrumentoSistema e InstrumentoPersonalizado
<code>src/Entity/InstrumentoMusico.php</code>	La relación ahora la gestiona Doctrine con ManyToMany
<code>src/Repository/InstrumentoRepository.php</code>	Ya no existe la entidad Instrumento
<code>src/Repository/InstrumentoMusicoRepository.php</code>	Ya no existe la entidad InstrumentoMusico

Ficheros modificados

`src/Entity/Musico.php`

Se eliminó la relación `OneToMany` a `InstrumentoMusico` y se sustituyó por dos relaciones `ManyToMany`:

```
// ANTES
#[ORM\OneToMany(targetEntity: InstrumentoMusico::class, mappedBy: 'musico',
cascade: ['remove'])]
private Collection $instrumentoMusicos;

// DESPUÉS
#[ORM\ManyToMany(targetEntity: InstrumentoSistema::class)]
#[ORM\JoinTable(name: 'musico_instrumento_sistema')]
private Collection $instrumentosSistema;

#[ORM\ManyToMany(targetEntity: InstrumentoPersonalizado::class)]
#[ORM\JoinTable(name: 'musico_instrumento_personalizado')]
private Collection $instrumentosPersonalizados;
```

El método `getInstrumentos()` se mantiene y sigue funcionando igual en todos los templates. Internamente ahora fusiona las dos colecciones:

```
public function getInstrumentos(): Collection
{
    return new ArrayCollection(array_merge(
        $this->instrumentosSistema->toArray(),
        $this->instrumentosPersonalizados->toArray()
    ));
}
```

src/Form/MusicoType.php

El campo `instrumentos` (un solo EntityType) se sustituyó por dos campos separados:

- `instrumentos_sistema` → EntityType de `InstrumentoSistema`, muestra todos los del sistema
- `instrumentos_personalizados` → EntityType de `InstrumentoPersonalizado`, filtrado por el usuario actual

Ambos tienen `display: contents` en su `attr` para que los checkboxes participen directamente en el layout flex del contenedor, sin que los divs intermedios generados por Symfony rompan los estilos.

El formulario recibe `InstrumentoPersonalizadoRepository` por inyección de dependencias en el constructor (Symfony lo autowirea automáticamente).

src/Controller/MusicoController.php

- **new():** en lugar de crear objetos `InstrumentoMusico` manualmente, se añaden directamente a las colecciones del músico:

```
foreach ($form->get('instrumentos_sistema')->getData() as $i) {
    $musico->getInstrumentosSistema()->add($i);
}
```

```
}
```

- **edit():** se limpian las colecciones con `clear()`, se hace flush, y se re-añaden las nuevas selecciones. El flush intermedio evita conflictos de clave única.
- **delete():** sin cambios; las tablas de relación se borran automáticamente por la FK `ON DELETE CASCADE`.

src/Controller/InstrumentoController.php

El endpoint AJAX `POST /instrumento/add` ahora guarda en `InstrumentoPersonalizado` en lugar de `Instrumento`. La lógica de deduplicación busca por nombre **y por usuario** (antes buscaba solo por nombre en toda la tabla):

```
$existente = $repo->findOneBy(['nombre' => $nombre, 'usuario' => $this->getUser()]);
```

Esto permite que dos usuarios distintos creen un instrumento con el mismo nombre sin conflicto.

src/DataFixtures/AppFixtures.php

Cambiado para usar `InstrumentoSistema` en lugar de `Instrumento`. Los fixtures solo cargan instrumentos del sistema.

Templates (`new.html.twig`, `edit.html.twig`, `_form.html.twig`)

La sección de instrumentos pasó de un solo `form_widget` a dos:

```
{# ANTES #}
<div class="perfil-instrumentos-container">
    {{ form_widget(form.instrumentos) }}
</div>

{# DESPUÉS #}
<div class="perfil-instrumentos-container">
    {{ form_widget(form.instrumentos_sistema) }}
    {{ form_widget(form.instrumentos_personalizados) }}
</div>
```

El nombre del campo en el script AJAX que añade checkboxes dinámicamente también se actualizó:

```
// ANTES
input.name = '{{ form.instrumentos.vars.full_name }}[]';

// DESPUÉS
input.name = '{{ form.instrumentos_personalizados.vars.full_name }}[]';
```

Los templates `show.html.twig` e `index.html.twig` **no necesitaron cambios** porque usan `musico.instrumentos`, que sigue funcionando igual.

La migración en detalle

La migración (`Version20260524120000`) preserva todos los datos existentes:

1. Crea `instrumento_sistema` e inserta los registros donde `usuario_id IS NULL` conservando los IDs originales.
2. Crea `instrumento_personalizado` e inserta los registros donde `usuario_id IS NOT NULL` conservando los IDs originales.
3. Crea `musico_instrumento_sistema` y migra las filas de `instrumento_musico` que apuntaban a instrumentos del sistema.
4. Crea `musico_instrumento_personalizado` y migra las filas que apuntaban a instrumentos personalizados.
5. Elimina `instrumento_musico` e `instrumento`.

La migración tiene `down()` completo para poder revertirla si es necesario.

Cómo incorporar estos cambios (para compañeros)

```
git pull
composer install
php bin/console doctrine:migrations:migrate
```

Solo si la base de datos está vacía (sin fixtures previas):

```
php bin/console doctrine:fixtures:load
```

Si usas Docker y `composer install` da error de permisos en caché:

```
# Dentro del contenedor (docker compose exec php bash)
rm -rf var/cache/
chown -R www-data:www-data var/
```

Punto de retorno (git)

Si algo falla y se quiere volver al estado anterior al refactoring:

```
git reset --hard e240a5e
```

Y ejecutar `php bin/console doctrine:migrations:migrate --down` o restaurar la base de datos desde un backup.